Software Architecture

This document explains the overall design of the diode-characterization system: what it does, how a measurement is configured, how it is run, what each settings file controls, and why there are several "engine" files and how each one is selected and driven.

---

1. The big idea

This is an **automated lab-bench measurement system** for characterizing diodes. It drives three Keithley instruments over GPIB (via PyVISA) and records the results to CSV, then rolls those CSVs up into a single pass/fail report in the test station's text format.

The three instruments:

Instrument	Role	Driver file
Keithley **4200-SCS**	Source-Measure Unit (SMU) — sources V or I, measures V & I	`keithley4200.py`
Keithley **590**	LCR meter — measures capacitance & conductance	`keithley590_LCR.py`
Keithley **707A**	Switching matrix — routes signals to the device pin under test	`keithley707a.py`

The **central loop** is always the same shape, regardless of measurement type:

```
for each row in the Excel parameter matrix:
    1. set the switching matrix to connect the right pin   (707A)
    2. apply source setpoints and let them settle          (4200 or 590)
    3. measure                                             (4200 or 590)
    4. append a result row
finally:
    safe shutdown (power off sources, open the matrix)
```

What *changes* between measurement types is **what gets sourced and measured in steps 2–3**. That variation is captured by the different engine classes (see §6).

Data flow at a glance

```
settings/*.json  ─┐
                  ├─►  load_*_spec()  ─►  frozen dataclass "spec"  ─►
Runner.run()
settings/*.xlsx  ─┘         (config)            (data model)            (engine)
                                                                          │
                                                                          ▼
```

```
results/<name>_<ts>.csv

run_all.py aggregates 4 CSVs + settings/run_info.json ─► ReportWriter ─►
results/result_*.txt
```

---

2. How a measurement is configured

A measurement is configured by **two inputs**:

1. **A JSON config** in `settings/` — *how* to measure: instrument addresses, source setpoints, compliance, speed, etc. One JSON file == one measurement type.
2. **An Excel workbook** (`settings/ParameterMatrix_DF.xlsx`) — *where* to measure: one row per device pin, giving the switching-matrix routing and the pin label. Each JSON config names which **sheet** of the workbook it reads.

So the JSON answers "what stimulus do I apply and how do I read it back," and the Excel sheet answers "which pins do I iterate over and how do I wire them up." The engine takes the cross product: for every row in the sheet, it runs the stimulus defined in the JSON.

---

3. The JSON settings files

All configs live in `settings/`. There are four measurement configs plus one run-metadata file.

File	Measurement	Engine selected
`guard_leakage.json`	Guard ring leakage current	`SweepRunner`
`dark_current.json`	Reverse/forward dark current	`SweepRunner`
`series_resistance.json`	Series resistance Rs	`SeriesResistanceRunner`
`capacitance.json`	Junction capacitance	`LCRRunner`
`run_info.json`	Wafer/lot/operator metadata for the report (not a measurement)	—

3.1 Common top-level keys

```
{
  "test_name": "Dark Current",        // used for CSV filename + report
  "excel_sheet": "DarkCurrent",       // which workbook sheet to iterate
  "instruments": {
    "switch_matrix": "GPIB0::18::INSTR",   // 707A VISA address
    "smu_mainframe": "GPIB0::17::INSTR",   // 4200 VISA address (SMU configs)
    "lcr_meter":     "GPIB0::15::INSTR",   // 590 VISA address (LCR config only)
    "matrix_settling_s": 0.3,              // relay settle delay after routing
    "workbook": "settings/ParameterMatrix_DF.xlsx"  // optional; overrides default
path
  }
```

```
  // ... then either "smus" or "lcr" (see below)
}
```

3.2 SMU configs (`smus` block)

Used by `guard_leakage.json`, `dark_current.json`, `series_resistance.json`.

Each entry under `"smus"` is one SMU channel. **The key encodes the role and the SMU id**: `"cathode_SMU1"` → role `cathode`, channel `SMU1`. The *role* becomes the CSV column prefix (`Cathode Current A`, etc.).

```
"cathode_SMU1": {
  "sweep_mode": "Voltage in V",   // or "Current in A" (default: Voltage in V)
  "compliance_A": 0.1,            // current/voltage compliance limit
  "speed": "Slow",               // 4200 integration speed
  "range": "Auto",               // measurement range
  "average": 1,                   // readings averaged per point
  "sweep_profile": [              // list of (setpoint, hold) steps
    {"voltage": -1.25, "hold_s": 1.0},
    {"voltage":  2.5,  "hold_s": 3.0}
  ]
}
```

Each `sweep_profile` is a loop axis — the single most important rule:

`SweepRunner` treats every channel's `sweep_profile` as one axis of a **nested sweep** (Cartesian product). For each Excel row it sweeps every combination of every channel's steps:

- **JSON declaration order = loop nesting order.** The **first** channel in the JSON is the **outermost** loop (varies slowest); the **last** channel is the **innermost** (varies fastest). All channels are applied in declaration order at every combination, each honoring its own `hold_s` — there is no implicit skipping, so the timing is exactly what the JSON declares.
- A channel with a **single** step is simply held at that value across the whole nest, so it contributes one point to its axis. A config where only one channel is multi-step therefore collapses to a plain 1-D sweep of that channel — which is exactly how `guard_leakage.json` and `dark_current.json` behave.
- **Any number** of multi-step channels is allowed; the total row count per pin is the product of the per-channel step counts. `load_measurement_spec()` no longer restricts this. (The `ChannelSpec.is_primary` helper — "more than one step" — still exists and is used by the series-resistance engine to locate its swept channel.)

`sweep_mode` lets a channel source current instead of voltage (used by series resistance, which forces two current setpoints). Despite the field being named `voltage`, it carries the current value in amps when `sweep_mode` is `"Current in A"`.

3.3 LCR config (`lcr` block)

Used by `capacitance.json`. No SMUs; instead an `"lcr"` block configures the 590:

```
"lcr": {
  "frequency": 100000000.0,      // measurement frequency (Hz)
  "integration": "10 /s",        // integration / reading rate
  "range": "Auto",
  "trigger": "One-shot, talk",
  "bias_voltage": 0.0            // DC bias applied during the C/G read
}
```

The presence of the `"lcr"` key is itself the dispatch signal (see §5).

3.4 `run_info.json`

Pure metadata — wafer id, lot id, operator, device position, temperature, test station. It is **not** a measurement config; it is consumed only by the report writer to fill in the report header and to build the output filename.

---

4. The Excel parameter matrix

`parameter_matrix.py` reads the workbook. `load_parameter_rows()`:

- Opens the workbook, selects the sheet named by `excel_sheet`.
- **Normalizes column names** (case- and whitespace-insensitive; e.g. `matrixconfig`, `Matrix Config`, `MATRIX CONFIG` all map to `Matrix Config`).
- Requires two columns: **`Matrix Config`** (the 707A crosspoint routing string) and **`Measured Pin`** (the pin label). Rows missing either are skipped with a console note.
- Returns a `List[Dict[str, str]]` — one dict per pin.

Each measurement type uses a **different sheet** of the same workbook, so the same physical file holds the pin list for guard leakage, dark current, capacitance, and series resistance independently.

The matrix routing string itself is canonicalized by `normalize_matrix_config()` in `switching_matrix.py` (uppercased, comma/semicolon-tolerant, joined with `;`).

---

5. How to run

Single measurement — `sweep.py`

```
python sweep.py settings/dark_current.json
python sweep.py settings/guard_leakage.json --dry-run
python sweep.py settings/capacitance.json --limit-rows 3
python sweep.py settings/series_resistance.json --output results/rs.csv
```

Flags:

Flag	Effect
`--dry-run`	Simulate without touching hardware or sleeping (uses `DryRunResourceManager` / `DryRunPort`, which print every driver command).
`--limit-rows N`	Process only the first N rows of the Excel sheet.
`--output PATH`	Override the CSV output path (default: `results/<TestName>_<timestamp>.csv`).

One device, no prober — `run_all.py`

```
python run_all.py
python run_all.py --dry-run --limit-rows 2
```

`run_all.py` runs the four configs **in a fixed order** (guard leakage → dark current → capacitance → series resistance), waiting a few seconds between tests, then feeds the four resulting CSVs plus `run_info.json` into `ReportWriter`. The reusable core is `run_device(run_info, *, dry_run, limit_rows, output_dir)`, which writes all four CSVs and the report into one **per-device folder** `results/wafer<id>/dev<NNN>_x<x>_y<y>/` (fixed, collision-free names). This is the **no-prober** path — `run_all.py` never imports the prober.

Full wafer — `run_wafer.py`

```
python run_wafer.py GPIB0::1::INSTR accretech/examples/DF.mdf
python run_wafer.py GPIB0::1::INSTR accretech/examples/DF.mdf --dry-run --limit-rows 1
```

`run_wafer.py` is the outer prober loop: it parses an `.mdf` probe plan (`accretech_prober.parsers.read_mdf_probe_plan` → `(die_x, die_y)` per `PROB` die), and for each die calls `controller.move_to_die(x, y)` then the **same** `run_device()` `run_all.py` uses — with `run_info` derived from `settings/run_info.json` (position = die x/y, auto- incrementing `device_no`). The prober is the standalone `accretech-prober` package (under `accretech/`, wired as a **uv editable path dependency**; not modified by this repo). `--dry-run` swaps in a `NullProberController` so the whole loop runs with no GPIB. On error it calls `abort_and_safe_state()`; it always `separate()`s and closes the prober in `finally`.

Dispatch logic (how the runner is chosen)

Both `sweep.py` and `run_all.py` load the JSON and pick an engine based on which keys are present. **No engine is named in the config; it is inferred:**

Key present in JSON	Engine class	Spec loader
`"lcr"`	`LCRRunner`	`load_lcr_spec`
`"series_resistance"`	`SeriesResistanceRunner`	`load_measurement_spec`

Key present in JSON	Engine class	Spec loader
neither (has `"smus"`)	`SweepRunner`	`load_measurement_spec`

All runners expose the **same interface**:

```
Runner(spec, dry_run, limit_rows, output_path).run() -> Path
```

That uniform contract is what lets `sweep.py`/`run_all.py` treat them interchangeably.

---

6. Why there are several engine files

The engines live in `core/` and exist because the three measurement families differ in **what happens inside the per-row loop** — the source/measure step — even though the surrounding scaffolding (load rows, connect, route matrix, shutdown) is identical. Rather than one giant branching loop, the variation is expressed through a small class hierarchy.

```
SweepRunner                 (core/engine.py)        ← generic V/I sweep
   └── SeriesResistanceRunner (core/series_resistance_engine.py)  ← overrides
_run_row only

LCRRunner                   (core/lcr_engine.py)    ← standalone, drives the 590
```

6.1 `SweepRunner` — `core/engine.py`

The generic engine and the base class. For each row it:

1. Routes the matrix to the row's crosspoints.
2. Walks the **Cartesian product** of all channels' sweep steps (`_build_combos`, built on `itertools.product` so the first channel is the outermost loop). At each combination it applies every channel's setpoint + hold in declaration order.
3. Measures **all** channels and appends **one CSV row per combination**, with column names generated from the channel roles (`Cathode Target V`, `Cathode Current A`, …). Each row carries a flattened global `Step Index` plus a per-channel `<Label> Step Index`.

This covers guard leakage and dark current: source a voltage on one channel, hold the others, read the current. With only one multi-step channel the product collapses to that channel's steps, and the global `Step Index` equals its step index — which is what `report.py` relies on (see §6.4 / the note in `report.py`).

6.2 `SeriesResistanceRunner` — `core/series_resistance_engine.py`

Subclasses `SweepRunner` and **overrides only `_run_row`**. It reuses all the connection, matrix, and shutdown logic from the base class. The difference:

- It sources **two current** setpoints (the primary is in `"Current in A"` mode).
- It measures V & I at each, then computes `Rs = ((V2 − V1) + 2·ln(I2/I1)·26mV) / (I2 − I1)`.

- It emits **one CSV row per pin** (the computed Rs), not one per step.

This is the textbook case for inheritance: same loop skeleton, different math at the measurement point.

6.3 `LCRRunner` — `core/lcr_engine.py`

A **standalone** runner (does *not* extend `SweepRunner`) because it drives an entirely different instrument (the 590, not the 4200) with a different port termination, a different configure/measure protocol, and no notion of multiple SMU channels. It keeps the same outer shape — load rows, connect, per-row loop, safe shutdown — and the same `.run() -> Path` contract, but the body sets a DC bias, triggers one C/G read, and records capacitance, conductance, and DC bias per pin.

It is a sibling rather than a subclass because almost nothing in the SMU-specific base would be reused; sharing it would mean fighting the inheritance rather than benefiting from it.

6.4 How the engine is "driven"

The engine is **driven by the spec dataclass**, which is in turn built from the JSON. The flow:

1. `sweep.py` reads the JSON, inspects its keys, and calls the matching `load_*_spec()`.
2. The loader (`core/channel.py` or `core/lcr_channel.py`) parses the JSON into a **frozen dataclass** (`MeasurementSpec` / `LCRMeasurementSpec`). This is where parsing and any validation live. Nothing mutable or hardware-related is in the data model.
3. The chosen Runner is constructed with that spec and `.run()` is called. The runner reads everything it needs (addresses, channels, sweep profile) off the spec — it never re-reads the JSON.

So: **JSON → loader → immutable spec → runner**. The runner's behavior is fully determined by the spec it is handed; swapping configs swaps behavior without touching engine code.

---

7. Supporting modules

Module	Responsibility
`core/channel.py`	`MeasurementSpec`/`ChannelSpec`/`HardwareConfig`/`SweepStep` dataclasses + `load_measurement_spec()` (SMU configs). Channels keep JSON declaration order, which the engine uses as sweep-nest order.
`core/lcr_channel.py`	`LCRMeasurementSpec`/`LCRSpec` dataclasses + `load_lcr_spec()` (LCR config).
`core/port.py`	`PortWrapper` adapts a PyVISA resource to the `.write/.read/.query` API the SweepMe! drivers expect. `DryRunPort`/`DryRunResourceManager` stub hardware for `--dry-run` and print every command.
`switching_matrix.py`	`SwitchingMatrix707A` wraps the 707A driver (`apply_route`, `open_all`, `shutdown`); `normalize_matrix_config()` canonicalizes crosspoint strings.
`parameter_matrix.py`	Reads/normalizes the Excel workbook into row dicts.
`core/report.py`	`ReportWriter` turns the four measurement CSVs + `run_info.json` into the station's tab-separated `.txt` result with per-pin pass/fail bins and

Module	Responsibility
	limits.
`core/post_process.py`	SweepMe!-style post-processing script (standalone; not called by `sweep.py`/`run_all.py`).
`keithley4200.py`, `keithley590_LCR.py`, `keithley707a.py`	The actual SweepMe! instrument drivers (GPIB command protocols).
`helper/*.py`	Standalone GPIB diagnostics (`ping_4200.py`, `deep_flush.py`, `bus_reset.py`, `test_4200_*.py`). Run directly.

8. Outputs

- **`sweep.py` (single measurement)** — `results/<TestName>_<timestamp>.csv`, one row per sweep-nest combination (or per pin for series resistance / capacitance), columns derived from channel roles, plus a global `Step Index` and per-channel `<Label> Step Index`.
- **`run_all.py` / `run_wafer.py` (per device)** — a folder `results/wafer<id>/dev<NNN>_x<x>_y<y>/` holding the four measurement CSVs (`guard_leakage.csv`, `dark_current.csv`, `capacitance.csv`, `series_resistance.csv`) and the device report `result_dev<NNN>.txt`. `run_wafer.py` creates one such folder per probed die.
- **Report** — tab-separated `result_dev<NNN>.txt`, with a metadata header (from `run_info.json`), one line per pin-test with applied limits and a pass/fail bin, and a footer with total time, soft bin, and overall pass flag.

9. Environment

- Requires **Python 3.9.x exactly** (`requires-python = "==3.9.*"`).
- Install with `uv sync`.
- Instruments communicate over **GPIB via PyVISA**; the drivers are SweepMe! device classes that depend on `pysweepme`.
- Use `--dry-run` to exercise the full code path with no hardware attached — every instrument command is printed instead of sent.